

گزارش پروژه سوم بینایی LiDAR ICP

استاد درس: دکتر جوانمردی، دکتر خادمیان

امیررضا کاظم لو

درس ربات‌های متحرک خودگردان

۲۲ خرداد ۱۴۰۵

۱ مقدمه

هدف از استفاده از الگوریتم ICP برای تخمین حرکت مبتنی بر LiDAR، یافتن پارامترهای تبدیل (انتقال t و چرخش R) است که به بهترین شکل نقاط متناظر در دو ابر نقطه (Point Cloud) را روی هم منطبق کند. از منظر ریاضی، این الگوریتم به دنبال یافتن پارامترهایی است که مجموع مربعات خطا را کمینه کند. به عبارت دیگر، هدف کمینه کردن تابع هزینه $E(R, t) = \sum_{(i,j) \in C} ||q_i - Rp_j - t||^2$ است.

در این پروژه، انتخاب دو اسکن LiDAR با یک فاصله زمانی کوچک اما قابل مشاهده (Small but Visible) Baseline) بین ۱ تا ۳ ثانیه الزامی است. از آنجایی که سنسور LiDAR با فرکانس 10 Hz داده ثبت می‌کند، یک فاصله ۱ تا ۳ ثانیه‌ای برابر با فاصله ۱۰ تا ۳۰ فریم متوالی ابر نقطه است. منظور از بیس‌لاین قابل مشاهده این است که پهناد باید جابجایی فیزیکی ملموسی بین دو اسکن انتخاب شده داشته باشد. اگر اسکن‌هایی با فاصله ۱.۰ ثانیه انتخاب شوند، تقریباً یکسان به نظر می‌رسند و ثبت (Registration) آن‌ها بسیار ساده و فاقد ارزش ارزیابی برای الگوریتم ICP است. در مقابل، اگر اسکن‌ها ۲۰ ثانیه فاصله داشته باشند، پهناد ممکن است به طور کامل از ساختار عبور کرده باشد و هیچ هندسه مشترکی برای تطابق باقی نماند. بنابراین، فاصله ۱ تا ۳ ثانیه تضمین می‌کند که یک جابجایی فیزیکی مشخص وجود دارد، در حالی که ساختار محیطی مشترک کافی برای تراز کردن موفقیت‌آمیز اسکن‌ها حفظ شده است. داده‌های مورد استفاده در این پروژه در فرمت PCD ذخیره شده‌اند. فرمت PCD مخفف Point Cloud Data است. این فرمت یک فایل استاندارد است که در ابتدا توسط توسعه‌دهندگان کتابخانه Point Cloud Library (PCL) به طور خاص برای ذخیره داده‌های ابر نقطه سه‌بعدی ثبت‌شده توسط سنسورهای مانند LiDAR، دوربین‌های عمق‌سنج یا اسکنرهای سه‌بعدی ایجاد شده است.

با بررسی نام فایل‌های داده شده، اگر زمان ثبت (که به صورت UNIX timestamp در نام فایل‌ها مشخص است) را از هم کم کنیم، تفاوت فایل 001 و 002 حدود 2.1 ثانیه است. همچنین تفاوت فایل‌های 003 و 004 حدود 1.2 ثانیه است. از آنجا که صورت پروژه نیازمند انتخاب دو اسکن از یک پرواز با فاصله ۱ تا ۳ ثانیه است، انتخاب هر دو فایل متوالی در این پوشه، نیازمندی‌های بخش اول پروژه (Task 1) را به طور کامل برآورده می‌کند.

۲ نحوه انجام

الگوریتم پایه ICP شامل یک حلقه تکرارشونده است که در هر مرحله، از طریق درخت‌های جستجو مانند kd-trees نقاط متناظر (Data Association) را تعیین می‌کند. سپس، ماتریس چرخش R و بردار انتقال t را محاسبه نموده و

این تبدیل را روی نقاط اعمال می‌کند تا زمانی که میزان خطا کاهش یافته و به زیر یک حد آستانه برسد. بر اساس مبانی نظری، الگوریتم پایه (Vanilla ICP) در صورتی به درستی همگرا می‌شود که یک حدس اولیه مناسب در دسترس باشد؛ در غیر این صورت بهینه‌سازی ممکن است در کمینه‌های محلی متوقف شود. به همین دلیل، برای دستیابی به بالاترین میزان تطابق (Fitness) و کمترین میزان خطای RMSE، از یک رویکرد دو مرحله‌ای درشت‌به‌ریز (Coarse-to-Fine) استفاده شده است.

- **(Preprocessing Settings):** به جای استفاده از تمامی نقاط که هزینه محاسباتی بالایی دارد، از تکنیک Voxel Grid Downsampling (نمونه‌برداری یکنواخت) استفاده شد. این کار علاوه بر افزایش کارایی، جستجو برای یافتن تناظرها را ساده‌تر می‌کند. در مرحله Coarse اندازه وکسل (voxel_size) برابر با 0.5 متر تنظیم شد تا ساختارهای کلی به سرعت استخراج شوند. در مرحله Fine این مقدار به 0.1 متر کاهش یافت تا جزئیات دقیق هندسی حفظ شوند. همچنین برای اجرای مرحله دوم، بردارهای نرمال سطح (Normals) برای هر نقطه تخمین زده شد.
- روش مقداردهی اولیه: برای مرحله اول، حدس اولیه به صورت یک ماتریس همانی 4×4 (Identity Matrix) تنظیم شد که به معنای فرض عدم حرکت پھیاد است. برای مرحله دوم، ماتریس تبدیل (Transformation Matrix) حاصل از خروجی مرحله اول، به عنوان حدس اولیه به الگوریتم داده شد تا تضمین‌کننده همگرایی دقیق باشد.
- پارامترهای ICP: در مرحله درشت (Coarse) از معیار خطای Point-to-Point استفاده شد که فاصله نقطه به نقطه را کمینه می‌کند. در این حالت، الگوریتم از تجزیه مقادیر منفرد (SVD) روی ماتریس کوواریانس متقاطع برای یافتن تبدیل بهینه استفاده می‌کند. آستانه شعاع جستجو (Threshold) در این مرحله 3.0 متر در نظر گرفته شد تا الگوریتم بتواند هم‌پوشانی‌های دورتر را پیدا کند.
- در مرحله (Fine)، الگوریتم به Point-to-Plane تغییر یافت. این معیار مجموع فواصل مربع بین یک نقطه و صفحه مماس (Tangent Plane) در نقطه متناظر آن را کمینه می‌کند. این روش به مناطق صاف اجازه می‌دهد روی هم بلغزند و معمولاً نرخ همگرایی بسیار بهتری نسبت به روش نقطه-به-نقطه دارد. آستانه جستجو در این مرحله به 1.5 متر کاهش یافت تا تطابق بسیار دقیقی بدون در نظر گرفتن نقاط پرت صورت گیرد.
- **Stopping Criteria:** حلقه تکرار الگوریتم زمانی متوقف می‌شود که تغییرات خطا از یک آستانه مشخص (error < threshold) کمتر شود یا تعداد تکرارها به حد نهایی برسد. برای جلوگیری از تکرارهای نامتناهی در شرایط عدم همگرایی، حداکثر تعداد تکرار در هر دو مرحله برابر با ۲۰۰۰ تنظیم گردید.

۳ توضیح کد و پیاده‌سازی

در این بخش، پیاده‌سازی الگوریتم و توابع استفاده شده در کتابخانه Open3D مورد بررسی قرار می‌گیرد:

- تابع کمکی (draw_registration_result): این تابع دو ابر نقطه و یک ماتریس تبدیل T را دریافت می‌کند. کپی‌های موقتی از ابر نقاط ایجاد می‌کند تا داده‌های اصلی تغییر نکنند. ابر نقطه source را زرد و ابر نقطه target را آبی رنگ می‌کند. در نهایت، تبدیل را روی ابر نقطه مبدأ اعمال کرده و یک پنجره سه‌بعدی برای نمایش همزمان آن‌ها باز می‌کند.
- بارگذاری داده‌ها (Data Loading): تابع o3d.io.read_point_cloud فایل‌های با پسوند pcd را از حافظه سیستم خوانده و به عنوان اشیاء ابر نقطه کتابخانه Open3D در حافظه بارگذاری می‌کند.

- پیش‌پردازش و کاهش نمونه‌برداری (**Voxel Downsampling**) : پردازش داده‌های خام LiDAR (که شامل صدها هزار نقطه است) از نظر محاسباتی بسیار سنگین است. تابع `voxel_down_sample` یک شبکه سه‌بعدی از مکعب‌ها (وکسل‌ها) با اندازه مشخص (به عنوان مثال ۵.۰ متر) ایجاد می‌کند. تمامی نقاط داخل یک وکسل میانگین‌گیری شده و به یک نقطه واحد تبدیل می‌شوند. این کار سرعت الگوریتم ICP را به شدت افزایش داده و نویز را کاهش می‌دهد.
- تخمین نرمال (**Normal Estimation**) : الگوریتم استاندارد ICP فواصل را به صورت نقطه-به-نقطه محاسبه می‌کند. با این حال، ICP نقطه-به-صفحه (Point-to-Plane) معمولاً سریع‌تر و مقاوم‌تر است. برای استفاده از آن، الگوریتم نیاز دارد جهت سطح (بردار نرمال) هر نقطه را بداند. تابع `estimate_normals` این مقادیر را با بررسی نقاط همسایه محاسبه می‌کند.
- مقداردهی اولیه (**Initialization**) : الگوریتم ICP یک الگوریتم بهینه‌سازی محلی است، به این معنی که برای عملکرد صحیح به یک حدس اولیه نیاز دارد. در این کد از `np.identity(4)` استفاده شده است که یک ماتریس همانی 4×4 است. این امر اساساً به الگوریتم می‌گوید: «به عنوان نقطه شروع، فرض کن پهنای بین اسکن ۱ و اسکن ۲ هیچ حرکتی نداشته است.»

۱.۳ هسته اصلی الگوریتم (ICP Registration)

تابع `registration_icp` هسته اصلی این پیاده‌سازی است. این تابع فاصله بین ابر نقاط مبدأ و مقصد را محاسبه کرده، یک تبدیل جدید برای نزدیک‌تر کردن آن‌ها می‌یابد و این کار را تا زمانی که به حد نهایی تکرارها برسد یا نقاط هم‌تراز شوند، تکرار می‌کند. پارامترهای این تابع به شرح زیر است:

- پارامترهای `source_down` و `target_down` : این پارامترها ابر نقاط ورودی هستند. `source_down` ابر نقطه‌ای است که می‌خواهیم حرکت دهیم و `target_down` ابر نقطه مرجع است که کاملاً ثابت می‌ماند. پسوند `_down` نشان می‌دهد که این‌ها، نسخه‌های کاهش‌یافته با وکسل از ابر نقاط اصلی هستند که استفاده از آن‌ها باعث اجرای بسیار سریع‌تر تابع می‌شود.
- آستانه فاصله (**threshold**) : در هر تکرار، ICP برای هر نقطه در ابر نقطه مبدأ، به دنبال نزدیک‌ترین نقطه در ابر نقطه مقصد می‌گردد. پارامتر آستانه (به عنوان مثال 1.0 متر) به الگوریتم می‌گوید که از جفت نقاطی که بیش از این فاصله از هم دور هستند، کاملاً چشم‌پوشی کند. این امر برای مقابله با نویز و بخش‌هایی از صحنه که فقط در یکی از اسکن‌ها وجود دارند، حیاتی است.
- حدس اولیه (**initial_guess**) : یک ماتریس تبدیل 4×4 است که نقطه شروع را به الگوریتم می‌دهد. اگر موقعیت شروع مناسب نباشد، الگوریتم در یک کمینه محلی گیر می‌کند. در بخش اول پروژه، از یک ماتریس همانی استفاده شد، اما در بخش دوم (Task 2)، حدس اولیه همان ماتریس تبدیل نهایی به دست آمده از جفت اسکن قبلی خواهد بود.
- استراتژی **TransformationEstimationPointToPlane** : روش محاسبه خطا را تعیین می‌کند. بر خلاف روش نقطه-به-نقطه (Point-to-Point) که سعی می‌کند نقاط را دقیقاً روی هم بیندازد و روی سطوح صاف دچار لغزش می‌شود، روش نقطه-به-صفحه (Point-to-Plane) فاصله نقطه مبدأ تا سطح صاف (صفحه مماس) مقصد را کمینه می‌کند. این روش به نقاط اجازه می‌دهد در امتداد دیوارهای صاف بلغزند تا بهترین تناسب پیدا شود و برای محیط‌های ساختمانی معماری بسیار دقیق‌تر است.

- شرط توقف : $ICPConvergenceCriteria(max_iteration=2000)$: ICP یک حلقه تکرارشونده است. تنظیم حداکثر تکرار روی ۲۰۰۰ یک محدودیت قطعی برای الگوریتم تعیین می‌کند تا در صورت عدم همگرایی کامل، در یک حلقه بی‌نهایت گیر نیفتد و بهترین تلاش خود را برگرداند.

۲.۳ معیارهای ارزیابی (Metrics)

برای سنجش دقت هم‌ترازی از دو معیار استفاده شده است:

- **Fitness** : میزان مساحت هم‌پوشانی را اندازه‌گیری می‌کند. این مقدار برابر با نسبت تناظرهای صحیح نسبت به کل تعداد نقاط است. عدد بالاتر نشان‌دهنده عملکرد بهتر است (حداکثر ۰.۱ است).
- خطای **Inlier RMSE** : ریشه میانگین مربعات خطای نقاطی است که با موفقیت تطبیق داده شده‌اند. عدد پایین‌تر به این معنی است که نقاط منطبق شده به یکدیگر نزدیک‌تر و از دقت بالاتری برخوردار هستند.

۴ نتایج بخش اول (Task 1 Results)

۱.۴ تلاش اولیه و نتایج نامطلوب

در تلاش اولیه برای هم‌ترازی دو ابر نقطه متوالی، الگوریتم با مقادیر پیش‌فرض اجرا شد. مقادیر ورودی در این حالت شامل اندازه وکسل برابر با ۰.۵، آستانه جستجو (Threshold) برابر با ۱.۰، نوع الگوریتم Point-to-Plane و حداکثر تکرار ۲۰۰۰ بود. نتایج خروجی این اجرا به صورت زیر به دست آمد:

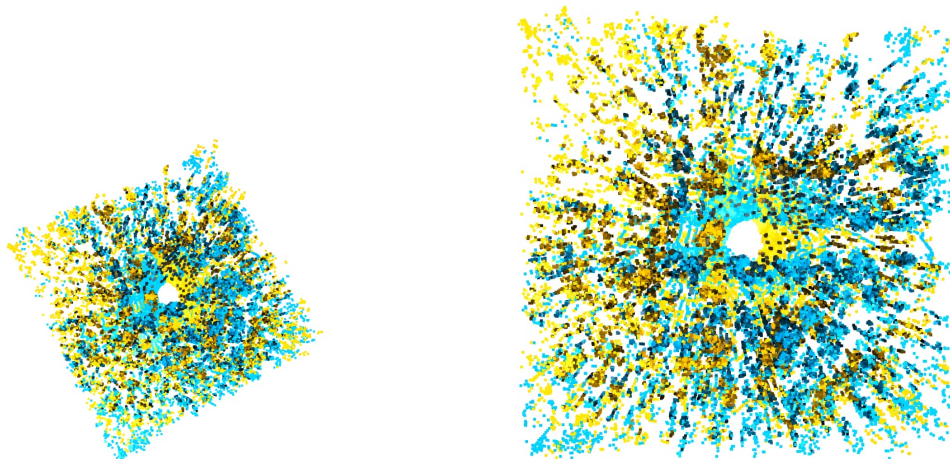
- امتیاز برازش (Fitness Score) : ۰.۳۴۸۰

- خطای **Inlier RMSE** : ۰.۶۲۹۹

- ماتریس تبدیل تخمین زده شده:

$$\begin{bmatrix} 0.9838 & -0.0020 & 0.1791 & -0.5736 \\ 0.0090 & 0.9992 & -0.0380 & -0.1611 \\ -0.1789 & 0.0390 & 0.9830 & -0.0461 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

همان‌طور که از نتایج پیداست (برازش حدود ۸.۳۴ درصد)، این مقادیر مطلوب نبودند. این موضوع نشان می‌دهد که الگوریتم در یک کمینه محلی (Local Minimum) متوقف شده است و هم‌پوشانی قابل قبولی بین نقاط ایجاد نشده است. در شکل ۱، وضعیت ابر نقاط قبل و بعد از این هم‌ترازی ناموفق نشان داده شده است.



شکل ۱: سمت راست: وضعیت ابر نقاط قبل از هم‌ترازی / سمت چپ: خروجی پس از هم‌ترازی با پارامترهای اولیه (عدم انطباق صحیح)

۲.۴ جستجوی شبکه‌ای پارامترها (Grid Search)

به دلیل نتایج ضعیف و عدم انطباق در اجرای اول، برای یافتن بهترین ترکیب از پارامترها از روش جستجوی شبکه‌ای (Grid Search) استفاده شد. در این جستجو، مقادیر زیر به صورت تو در تو مورد بررسی قرار گرفتند:

- **voxel_size**: [0.5, 0.3, 0.1]
- **threshold**: [1.0, 2.0, 3.0]
- **max_iter**: [500, 1000, 2000]
- **type_of_icp**: [point_to_point, point_to_plane]

توضیح منطق کد ارزیابی بهترین نتیجه

برای ارزیابی و یافتن بهترین نتیجه در طول حلقه‌های تو در تو، کد به صورت زیر عمل می‌کند:

- **best_fitness = -1.0**: از آنجا که امتیاز برازش یک درصد (بین 0.0 تا 1.0) است، مقدار پایه زیر صفر تنظیم می‌شود تا در همان تکرار اول، این متغیر بازنویسی شده و تبدیل به «بهترین فعلی» شود.
- **best_rmse = float('inf')**: مقدار اولیه خطا روی بی‌نهایت (Infinity) تنظیم می‌شود.
- شرط **if** برای انتخاب برنده: در هر ترکیب، کد می‌پرسد: «آیا مساحت هم‌پوشانی (برازش) از بهترین مقداری که تا الان دیده‌ام بیشتر است؟» اگر پاسخ مثبت باشد، متغیرها را به‌روزرسانی می‌کند. در صورتی که امتیازهای برازش کاملاً برابر باشند، از کمترین مقدار Inlier RMSE به عنوان عامل تعیین‌کننده (Tiebreaker) برای انتخاب برنده استفاده می‌شود.

۳.۴ بهترین نتایج یافت شده (Best Parameters Found)

پس از اجرای جستجوی شبکه‌ای، بهترین خروجی یافت شده به شرح زیر است:

۴.۴ Ablation Study

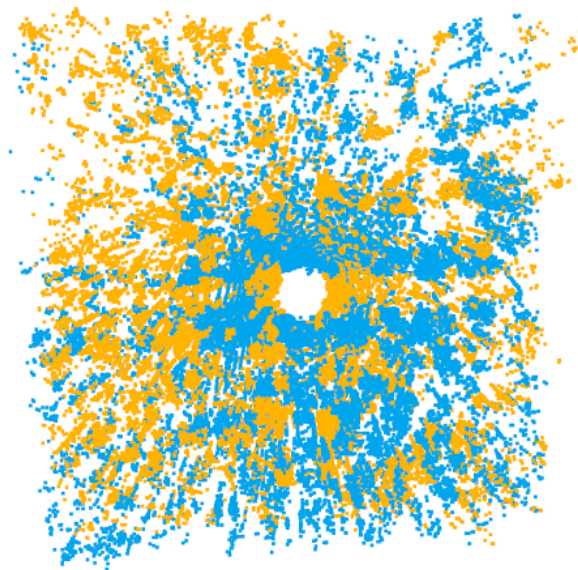
به منظور درک عمیق تر و بررسی تاثیر هر یک از پارامترهای پیش پردازش و الگوریتم ICP بر روی خروجی نهایی، یک مطالعه حذف (Ablation Study) در قالب جستجوی شبکه‌ای (Grid Search) انجام شد. در این آزمایش، حالت‌های مختلف اندازه و کسل (0.1, 0.3, 0.5)، آستانه جستجو (1.0, 2.0, 3.0)، حداکثر تعداد تکرار (500, 1000, 2000) و نوع الگوریتم (Point-to-Plane و Point-to-Point) مورد ارزیابی قرار گرفتند. با تحلیل نتایج به دست آمده از اجرای تمامی ترکیبات ممکن، الگوهای بسیار مهمی در رفتار الگوریتم مشاهده شد که به تفکیک هر پارامتر در ادامه بررسی می‌شوند:

- تاثیر (Threshold) : مهم‌ترین عامل در تغییر نتایج، پارامتر آستانه بود. با افزایش آستانه از 1.0 به 3.0 متر، میزان برازش (Fitness) به شدت افزایش یافت (از حدود 0.34 به 0.87). با این حال، همان‌طور که انتظار می‌رفت، این افزایش برازش به قیمت افزایش خطای RMSE (از 0.62 به 1.43 متر) تمام شد. این موضوع نشان می‌دهد که آستانه بزرگتر اجازه می‌دهد نقاط بیشتری جفت شوند، اما لزوماً تمامی این جفت‌ها از نظر فیزیکی دقیق نیستند.
- تاثیر (Voxel Size) : کاهش اندازه و کسل از 0.5 به 0.1 متر تاثیر ظریف‌تری بر نتایج داشت. در آستانه‌های بالاتر (مانند 3.0)، کاهش اندازه و کسل باعث شد تا الگوریتم لبه‌ها و جزئیات هندسی بیشتری را در نظر بگیرد که در نهایت منجر به یافتن بهترین خروجی کل در $\text{voxel_size} = 0.1$ گردید.
- تاثیر (Max Iterations) : مقایسه نتایج در تکرارهای 500، 1000 و 2000 نشان می‌دهد که در مقادیر یکسان سایر پارامترها، خروجی‌ها (اعم از Fitness و RMSE) کاملاً یکسان بوده‌اند. این پدیده اثبات می‌کند که الگوریتم بسیار پیش‌تر از 500 تکرار به همگرایی (یا یک کمینه محلی) رسیده و متوقف شده است. بنابراین افزایش این پارامتر بیش از حد نیاز، تنها بار محاسباتی را بدون بهبود نتیجه افزایش می‌دهد.
- نوع الگوریتم (Point-to-Plane در برابر Point-to-Point) : در این مرحله از هم‌ترازی (مرحله Coarse که فاصله‌ها و انحرافات اولیه زیاد است)، الگوریتم Point-to-Point به طور کلی مقادیر برازش کمی بالاتری نسبت به Point-to-Plane تولید کرد (برای مثال 0.876 در برابر 0.857 در بهترین حالت). این موضوع با مبانی نظری مطابقت دارد؛ زیرا وقتی ابرها دور هستند، تخمین سطح و لغزیدن روی صفحات ممکن است به دلیل جهت‌گیری اشتباه بردارهای نرمال، بهینه‌سازی را دچار خطا کند.
- در نهایت، با مقایسه تمامی ترکیبات، بهترین پارامترها برای مرحله درشت (Coarse) به صورت زیر استخراج گردید. این خروجی به عنوان حدس اولیه و ورودی قدرتمند برای مرحله دقیق (Fine) مورد استفاده قرار خواهد گرفت:
- بهترین پارامترها: $\text{type_of_icp} = \text{point_to_point}$, $\text{max_iter} = 500$, $\text{threshold} = 3.0$, $\text{voxel_size} = 0.1$
- بهترین امتیاز برازش (Best Fitness) : 0.8767
- کمترین خطای (Best Inlier RMSE) : 1.4346

• بهترین ماتریس تبدیل:

$$\begin{bmatrix} 0.9857 & 0.0740 & 0.1511 & 3.0773 \\ -0.0711 & 0.9971 & -0.0246 & 1.3062 \\ -0.1525 & 0.0135 & 0.9882 & -0.9024 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

همان طور که مشاهده می‌شود، با تنظیم صحیح پارامترها (به ویژه افزایش آستانه جستجو به ۰.۳ متر)، میزان هم‌پوشانی با جهش چشمگیری از ۳۴ درصد به ۶.۸۷ درصد رسیده است که نشان از هم‌ترازی ساختاری موفق دارد. در شکل ۲ خروجی مربوط به بهترین پارامترهای یافت‌شده در جستجوی شبکه‌ای نشان داده شده است.



شکل ۲: خروجی هم‌ترازی با بهترین پارامترهای جستجوی شبکه‌ای

با بررسی دقیق‌تر این خروجی و معیارهای ارزیابی، به نتایج زیر می‌رسیم:

- نقطه قوت (امتیاز برآزش = 0.876) : امتیاز برآزش از حدود ۳۴ درصد به ۶.۸۷ درصد جهش پیدا کرد. این بدان معناست که الگوریتم با موفقیت هندسه هم‌پوشان را برای اکثریت قریب به اتفاق نقاط مبدأ پیدا کرده است. با نگاهی به تصویر، ابرهای زرد و آبی اکنون به جای اینکه در کنار هم قرار بگیرند، فضای عمومی یکسانی را اشغال کرده‌اند.
- نقطه ضعف (خطای $\text{Inlier RMSE} = 1.43$) : این نقطه ضعف اصلی این اجرای خاص است. خطای RMSE برابر با 1.43 متر به این معنی است که فاصله متوسط بین نقاط تطبیق‌یافته زرد و آبی تقریباً یک و نیم متر است. برای نقشه‌برداری هوایی LiDAR، معمولاً انتظار می‌رود این عدد بین 0.1 تا 0.3 متر باشد.
- بررسی بصری : به دلیل بالا بودن خطای RMSE، تصویر کمی «مات» (Fuzzy) یا به طور مصنوعی ضخیم به نظر می‌رسد. دیوارها و تنه درختان به خطوط واحد و کاملاً مشخص تبدیل نشده‌اند؛ بلکه هم‌پوشانی دارند اما کمی تار و پخش شده‌اند.

دلیل این اتفاق چیست؟

کد جستجوی شبکه‌ای دقیقاً همان کاری را انجام داد که از آن خواسته شده بود: پارامترهایی را انتخاب کرد که منجر به بالاترین امتیاز برازش ممکن می‌شدند و یک آستانه ۰.۳ متری این هدف را محقق کرد. با این حال، یک آستانه بزرگ مانند یک شمشیر دو لبه عمل می‌کند:

- این کار یک تور بسیار بزرگ پهن می‌کند و به الگوریتم اجازه می‌دهد حتی اگر حدس اولیه بسیار دور از واقعیت باشد، تطابق‌ها را پیدا کند.
- از آنجایی که الگوریتم هر نقطه‌ای در شعاع ۳ متری را به عنوان یک «تطابق معتبر» در نظر می‌گیرد، هندسه‌های نادرست زیادی را با هم جفت می‌کند (به عنوان مثال، تطبیق یک نقطه روی شاخه درخت با نقطه‌ای روی زمین در ۲ متر پایین‌تر). این کار به طور مصنوعی امتیاز برازش را بالا می‌برد در حالی که دقت (RMSE) را از بین می‌برد.

۵.۴ رویکرد درشت‌به‌ریز (Coarse-to-Fine Registration)

برای بهره‌مندی از مزایای هر دو حالت (برازش بالا و خطای RMSE زیر سانتی‌متر)، رویکرد استاندارد صنعتی استفاده از خط لوله ثبت درشت‌به‌ریز (Coarse-to-Fine) است. در این روش به جای یک بار اجرای ICP، آن را دو بار متوالی اجرا می‌کنیم:

- مرحله درشت (Coarse Pass): اجرای ICP با آستانه بزرگ (مانند 3.0) و اندازه وکسل بزرگ (مانند 0.5) برای هم‌ترازی تقریبی ابرها (این اساساً همان چیزی است که خروجی قبلی به دست آورد).
- مرحله دقیق (Fine Pass): گرفتن ماتریس تبدیل از مرحله درشت و استفاده از آن به عنوان حدس اولیه (initial_guess) برای اجرای دوم. اما این بار آستانه را به 0.2 کاهش داده و از نوع point_to_plane استفاده می‌کنیم تا سطوح صاف کاملاً به طور محکم به هم بچسبند.

در اجرای اول این روش با ورودی $\text{threshold_fine} = 0.2$ ، خروجی‌های زیر به دست آمد:

--- STAGE 1: COARSE ALIGNMENT ---

Coarse Fitness: 0.8617

Coarse Inlier RMSE: 1.5171

--- STAGE 2: FINE ALIGNMENT ---

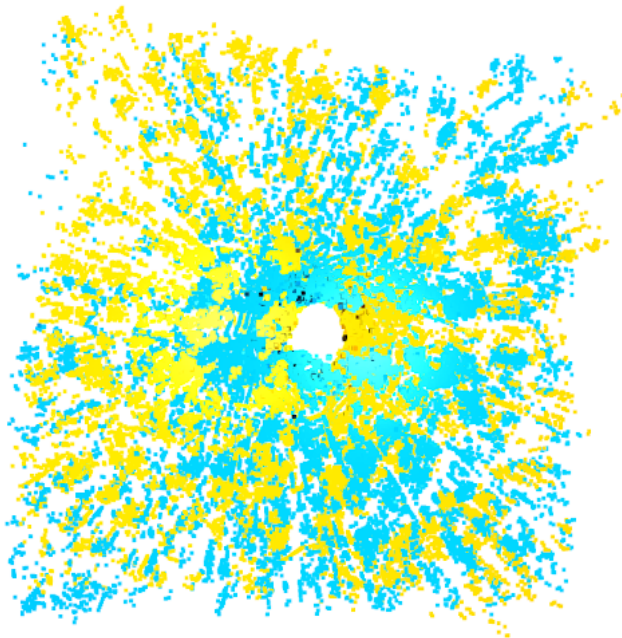
Fine Fitness: 0.0241

Fine Inlier RMSE: 0.1468

Final Optimal Transformation Matrix:

```
[[ 9.83930428e-01  9.81834433e-02  1.49133916e-01  2.57739203e+00]
 [-9.71742054e-02  9.95168114e-01 -1.40569940e-02 -6.66318121e-01]
 [-1.49793482e-01 -6.60865725e-04  9.88717086e-01 -7.19706439e-01]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00  1.00000000e+00]]
```


همان‌طور که در شکل ۳ مشاهده می‌شود، این نتیجه مطلوب نیست و میزان برازش در مرحله دوم به شدت (تا حدود ۴.۲ درصد) کاهش یافته است.



شکل ۳: خروجی نامطلوب هم‌ترازی با آستانه مرحله دقیق 0.2

برای حل این مشکل، مقدار آستانه مرحله دقیق به $\text{threshold_fine} = 1.5$ تغییر یافت تا با خطای RMSE مرحله اول متناسب باشد. خروجی نهایی و موفقیت‌آمیز این اصلاح به صورت زیر است:

--- STAGE 1: COARSE ALIGNMENT ---

Coarse Fitness: 0.8617

Coarse Inlier RMSE: 1.5171

--- STAGE 2: FINE ALIGNMENT ---

Fine Fitness: 0.5693

Fine Inlier RMSE: 0.8595

Final Optimal Transformation Matrix:

```
[[ 0.98320023  0.04252532  0.17750745  1.57369085]
 [-0.03769774  0.99882415 -0.03048264 -1.16666553]
 [-0.17859501  0.02327891  0.98364725 -0.53407007]
 [ 0.          0.          0.          1.          ]]
```

۶.۴ اعمال دستی تبدیل‌ها و ساخت نقشه (Manual Transformation & Mapping)

اگرچه استفاده از توابع آماده کتابخانه Open3D برای بخش ثبت و تخمین ماتریس تبدیل (ICP Registration) مجاز هست بر اساس اونچیزی که در گروه درس مطرح شده است، ولی اعمال این تبدیل‌ها روی ابر نقاط و همچنین جمع‌آوری آن‌ها برای ساخت نقشه سراسری باید به صورت دستی و با استفاده از محاسبات ماتریسی مستقیم پایتون (NumPy) انجام شود. در این بخش، مبانی ریاضی و پیاده‌سازی این فرآیند شرح داده شده است. این بخش از کد در فایل جداگانه solution with out open3d.ipynb انجام شده است.

الف) فرمولاسیون ریاضی جابه‌جایی دستی نقاط (Manual Transformation)

برای اعمال ماتریس تبدیل همگن 4×4 (شامل ماتریس چرخش $R_{3 \times 3}$ و بردار انتقال $t_{3 \times 1}$) بر روی یک فریم از ابر نقطه، ابتدا مختصات دکارتی هر نقطه $P_i = [x, y, z]^T$ به مختصات همگن $P_{i, \text{hom}} = [x, y, z, 1]^T$ تبدیل می‌شود. ماتریس تبدیل همگن T به صورت زیر تعریف می‌گردد:

$$T = \begin{bmatrix} R_{3 \times 3} & t_{3 \times 1} \\ 0_{1 \times 3} & 1 \end{bmatrix}$$

حاصل ضرب ماتریسی برای انتقال نقاط در مختصات همگن به صورت $P'_i = T \times P_{i, \text{hom}}$ محاسبه می‌شود. جهت پیاده‌سازی بهینه و برداری برای کل ماتریس نقاط به ابعاد $N \times 4$ ، محاسبات در پایتون با بهره‌گیری از کتابخانه NumPy به صورت ضرب ماتریسی زیر انجام می‌گیرد:

$$P_{\text{new}} = (T \times P^T)^T$$

پس از اتمام ضرب، مختصات همگن (ستون چهارم آرایه) حذف شده و نقاط مجدداً به فضای سه‌بعدی دکارتی $N \times 3$ بازمی‌گردند. همچنین در صورتی که ابر نقطه دارای بردارهای نرمال سطح (Normals) باشد، این بردارها جهت حفظ راستای هندسی خود تنها تحت تاثیر چرخش قرار می‌گیرند و بردار انتقال روی آن‌ها بی‌تاثیر است:

$$N_{\text{new}} = N \times R^T$$

ب) جمع‌بندی دستی فریم‌ها و ساخت نقشه (Manual Merging)

به جای استفاده از عملگر پیش‌فرض + در کتابخانه Open3D که به صورت خودکار دو شیء ابر نقطه را ادغام می‌کند، در رویکرد دستی آرایه‌های NumPy مربوط به مختصات نقاط استخراج شده و با استفاده از تابع np.vstack به صورت عمودی روی یکدیگر انباشته می‌شوند تا ساختار نقشه محلی شکل گیرد. در نهایت، آرایه نهایی ادغام‌شده مجدداً به یک شیء استاندارد o3d.geometry.PointCloud برای مصورسازی تبدیل می‌گردد.

ج) پیاده‌سازی کد به‌روزرسانی شده در بخش اول

کد زیر جایگزین شده است که در آن توابع manual_transform_point_cloud و manual_merge_point_clouds به طور کامل پیاده‌سازی و در تابع نمایش نتایج ادغام شده‌اند:

```
import open3d as o3d
import numpy as np
import copy
import os
```

```

def manual_transform_point_cloud(pcd, transformation_matrix):
    """
    Manually applies a 4x4 rigid transformation matrix to an Open3D point cloud using NumPy.
    """
    # 1. Extract Nx3 points to a NumPy array
    points = np.asarray(pcd.points)

    # 2. Convert to homogeneous coordinates (Nx4) by appending 1s
    ones = np.ones((points.shape[0], 1))
    points_homogeneous = np.hstack((points, ones))

    # 3. Apply the transformation matrix:  $P_{new} = T * P_{old}^T$ 
    transformed_points_homogeneous = transformation_matrix @ points_homogeneous.T

    # 4. Transpose back to Nx4 and drop the homogeneous coordinate to get Nx3
    transformed_points = transformed_points_homogeneous.T[:, :3]

    # 5. Assign the new points to a deep copy of the point cloud
    pcd_transformed = copy.deepcopy(pcd)
    pcd_transformed.points = o3d.utility.Vector3dVector(transformed_points)

    # 6. Manually rotate normals if they exist
    if pcd.has_normals():
        normals = np.asarray(pcd.normals)
        rotation_matrix = transformation_matrix[:3, :3] # Extract 3x3 rotation
        transformed_normals = normals @ rotation_matrix.T
        pcd_transformed.normals = o3d.utility.Vector3dVector(transformed_normals)

    return pcd_transformed

def manual_merge_point_clouds(pcd1, pcd2):
    """
    Manually concatenates two point clouds by stacking their NumPy arrays.
    """
    points1 = np.asarray(pcd1.points)
    points2 = np.asarray(pcd2.points)
    merged_points = np.vstack((points1, points2))

    merged_cloud = o3d.geometry.PointCloud()

```

```

merged_cloud.points = o3d.utility.Vector3dVector(merged_points)
return merged_cloud

def draw_registration_result(source, target, transformation, name_of_file="alignn

source_temp = copy.deepcopy(source)
target_temp = copy.deepcopy(target)

source_temp.paint_uniform_color([1, 0.706, 0])
target_temp.paint_uniform_color([0, 0.651, 0.929])

# --- TA REQUIREMENT: Manual Transformation ---
source_temp = manual_transform_point_cloud(source_temp, transformation)

o3d.visualization.draw_geometries([source_temp, target_temp])

# --- TA REQUIREMENT: Manual Map Building ---
merged_cloud = manual_merge_point_clouds(source_temp, target_temp)
o3d.io.write_point_cloud(f"{name_of_file}.pcd", merged_cloud)

# 1. Define paths to your specific files
cloud_1 = "./sampled_clouds_for_icp/001_cloud_1683269379_452476000_icp_sampled.p
cloud_2 = "./sampled_clouds_for_icp/002_cloud_1683269381_551481000_icp_sampled.p

# 2. Load the point clouds
source = o3d.io.read_point_cloud(cloud_1)
target = o3d.io.read_point_cloud(cloud_2)

# 3. Preprocessing (Voxel Downsampling)
voxel_size = 0.5
source_down = source.voxel_down_sample(voxel_size)
target_down = target.voxel_down_sample(voxel_size)

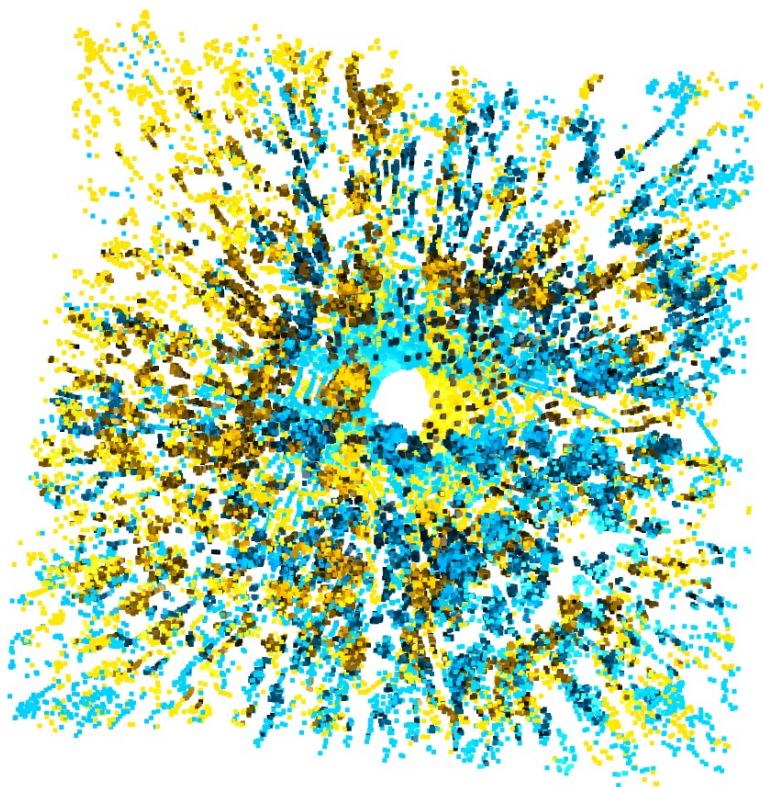
# Estimate normals (Required for Point-to-Plane ICP)
source_down.estimate_normals(o3d.geometry.KDTreeSearchParamHybrid(radius=voxel_s
target_down.estimate_normals(o3d.geometry.KDTreeSearchParamHybrid(radius=voxel_s

# 4. Initialization
initial_guess = np.identity(4)

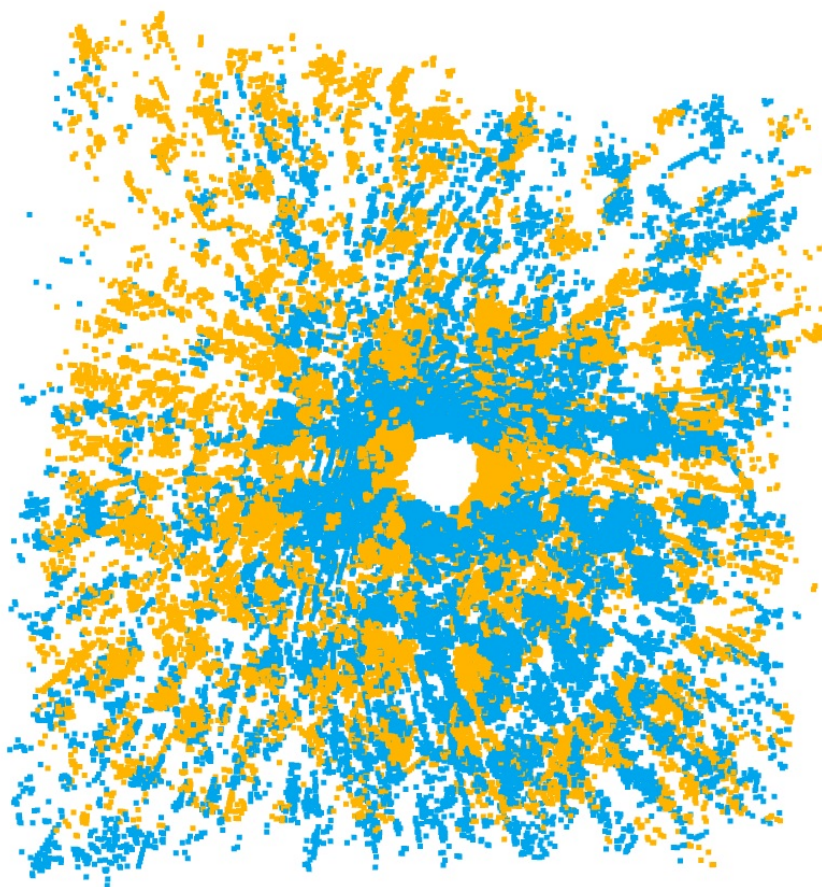
print("Visualizing Before Alignment (Source=Yellow, Target=Blue)...")

```

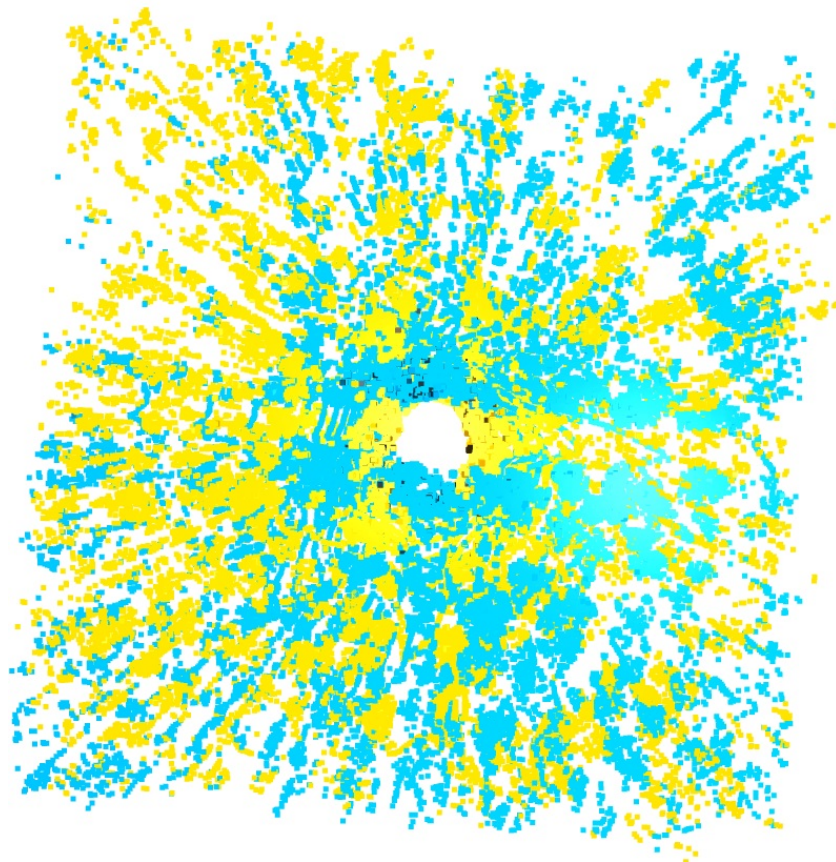
```
draw_registration_result(source_down, target_down, initial_guess, name_of_file="")
```



شکل ۴: خروجی حاصل از محاسبات و رسم manual با تنظیمات ترشولد یک و وکسل سایز ۵.۰



شکل ۵: خروجی حاصل از محاسبات و رسم manual با تنظیمات ترشولد ۳ و وکسل سایز ۱.۰ و نوع point to point



شکل ۶: خروجی حاصل از محاسبات و رسم manual با تنظیمات ترشولد ۳ و وکسل سایز ۵۰ و سپس با ترشولد ۲ و وکسل سایز ۲

۵ بخش دوم: Task 2: Consecutive Matching and Local Mapping

در این بخش، از تراز کردن تک اسکن فراتر رفته و به مبحث اندازه گیری مسافت طی شده (Odometry) و نقشه سازی محلی مبتنی بر LiDAR می پردازیم. هدف، ثبت یک توالی متوالی از اسکن ها، ترکیب تبدیل های دو به دو برای تخمین مسیر حرکت پهنپاد (UAV Trajectory) و تجمیع آن ها در یک نقشه سه بعدی واحد است.

۱.۵ مبانی ریاضی: ترکیب تبدیل ها

هنگامی که اسکن فعلی به اسکن قبلی متصل می شود، الگوریتم ICP یک ماتریس تبدیل نسبی برای ما محاسبه می کند. با این حال، برای ساخت یک نقشه یکپارچه، هر اسکن باید به سیستم مختصات اولین اسکن (که به عنوان قاب مرجع یا Global Frame در نظر گرفته می شود) منتقل شود. برای یافتن موقعیت مطلق هر اسکن در نقشه سراسری، باید حرکت نسبی آن را در موقعیت سراسری اسکن قبلی ضرب ماتریسی کنیم. فرمول این ترکیب به صورت زیر است:

$$T_{\text{global_current}} = T_{\text{global_prev}} \times T_{\text{relative_current}}$$

با استخراج مختصات انتقال، یعنی عناصر ستون چهارم ماتریس انتقال در طول مسیر، می‌توانیم مختصات دقیق فضایی حرکت پهپاد را استخراج کرده و مسیر آن را رسم کنیم.

۲.۵ تفاوت‌های کلیدی با بخش اول

- تغییر پویای هدف (**Dynamic Target Swapping**): بر خلاف بخش اول که مرجع کاملاً ثابت بود، در اینجا در انتهای هر تکرار از حلقه، اسکن فعلی به عنوان مرجع (هدف) برای اسکن بعدی قرار می‌گیرد.
- ضرب ماتریسی (**Matrix Dot Product**): هسته اصلی محاسبه موقعیت پهپاد، استفاده از ضرب ماتریسی برای اعمال تبدیل نسبی جدید روی موقعیت سراسری قبلی است تا موقعیت مطلق محاسبه شده و خطای انباشته (Drift) نیز در نظر گرفته شود.
- مدیریت حافظه (**RAM Management**): ترکیب تمامی ابر نقاط منجر به تولید میلیون‌ها نقطه می‌شود که به سرعت حافظه سیستم (RAM) را پر می‌کند. برای جلوگیری از این مشکل، نقشه سراسری در فواصل مشخص (مثلاً هر ۵ فریم) به صورت دوره‌ای با استفاده از روش Voxel Downsampling کاهش نمونه‌برداری می‌شود تا نقاط هم‌پوشان ادغام شوند.
- مصورسازی مسیر (**LineSet Visualization**): برای نمایش مسیر حرکت، از ساختار هندسی LineSet در کتابخانه Open3D استفاده شده است که خطوطی را به صورت متوالی بین مختصات‌های استخراج‌شده رسم می‌کند تا مسیر پرواز به وضوح قابل مشاهده باشد.

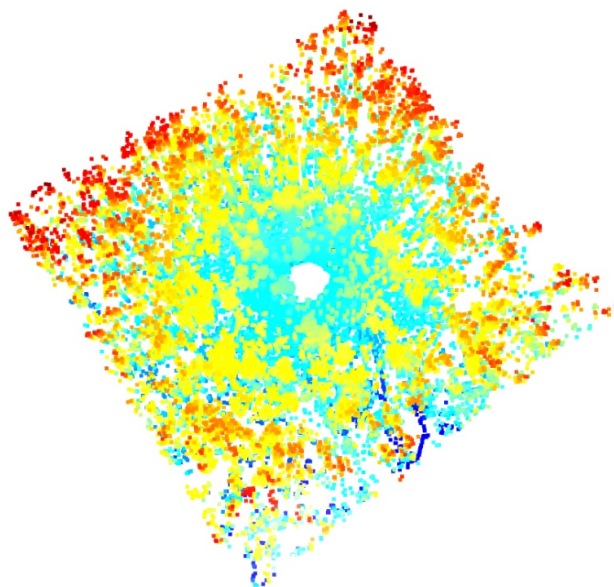
۳.۵ توضیح کد پیاده‌سازی

خط لوله (Pipeline) پیاده‌سازی شده برای این بخش شامل مراحل زیر است:

- بارگذاری خودکار فایل‌ها (**Dynamic Loading**): با استفاده از کتابخانه‌های سیستم عامل مانند glob، تمامی فایل‌های pcd موجود در پوشه به ترتیب زمانی خوانده شده و مرتب می‌شوند تا نیازی به وارد کردن دستی نام فایل‌ها نباشد.
- ایجاد نقشه و مسیر اولیه: اولین اسکن به عنوان مبدأ مختصات با ماتریس همانی مقداردهی شده و نقشه سراسری (global_map) با این اسکن آغاز می‌شود. همچنین نقطه شروع در آرایه مسیر پرواز ثبت می‌گردد.
- حلقه تکرار (**Iterative Loop**): در هر مرحله از حلقه، اسکن فعلی با استفاده از رویکرد درشت‌به‌ریز (Coarse-to-Fine) که در بخش قبل پارامترهای آن بهینه شد، با اسکن قبلی خود تراز می‌شود.
- اعمال تبدیل و تجمیع: پس از محاسبه ماتریس تبدیل سراسری جدید، این ماتریس روی ابر نقطه فعلی اعمال شده و هندسه جدید به نقشه سراسری افزوده می‌شود.
- ذخیره‌سازی و نمایش: در نهایت، نقشه یکپارچه کاهش‌یافته در یک فایل جدید به نام task2_accumulated_map.pcd ذخیره شده و به همراه خطوط قرمزرنگی که نشان‌دهنده مسیر پرواز پهپاد هستند، به صورت سه‌بعدی در خروجی نمایش داده می‌شود.

۴.۵ خروجی و مصورسازی نقشه نهایی

در شکل ۷ نمایی از نقشه سه بعدی نهایی که از تجمیع ۵۰ اسکن متوالی به دست آمده است، نمایش داده شده است. در این خروجی، مسیر تخمینی پرواز پهپاد نیز روی نقشه محلی قابل مشاهده است.



شکل ۷: خروجی نقشه محلی تجمیع شده و مسیر تخمینی پهپاد در بخش دوم

۶ بخش سوم: بررسی عوامل خطا

در این بخش، منابع اصلی خطا در الگوریتم ICP و تاثیر پارامترهای مختلف بر کیفیت نقشه سازی و ثبت داده ها مورد بحث و بررسی قرار می گیرد.

۱.۶ بررسی منابع اصلی خطا

- خطای انباشته یا دریفت (**Drift**): در بخش دوم پروژه (تطابق متوالی)، ماتریس موقعیت سراسری از ضرب متوالی ماتریس های تبدیل نسبی به دست می آید. از آنجا که هر اجرای ICP دارای مقدار کمی خطای ذاتی (نشان داده شده با RMSE) است، این خطاهای کوچک در طول زمان با یکدیگر جمع می شوند. به این پدیده Drift می گویند. بدون استفاده از تکنیک های جبرانساز مانند بهینه سازی گراف و حلقه بندی (Loop Closure)، مسیر تخمینی پهپاد در پروازهای طولانی به تدریج از واقعیت منحرف خواهد شد.
- فاصله بین اسکن ها (**Scan Spacing**): فاصله زمانی بین دو اسکن (بیس لاین ۱ تا ۳ ثانیه) نقش حیاتی در موفقیت الگوریتم دارد. اگر فاصله بسیار کم باشد، جابجایی قابل تشخیصی رخ نداده و الگوریتم دچار (**Overfitting**) روی نویز سنسور می شود. از سوی دیگر، اگر فاصله بسیار زیاد باشد، میزان (**Overlap**) بین دو

ابر نقطه به شدت کاهش یافته و الگوریتم نمی‌تواند نقاط متناظر را پیدا کند که منجر به شکست کامل عملیات Registration می‌شود.

- **(Voxel Size):** کاهش نمونه‌برداری با استفاده از Voxel Grid یک مصالحه (Trade-off) بین سرعت و دقت است. وکسل‌های بزرگ (مانند 0.5 متر) سرعت محاسبات را بالا می‌برند اما با از بین بردن جزئیات هندسی (مثل لبه دیوارها)، باعث کاهش دقت تطابق نهایی می‌شوند. به همین دلیل استفاده از رویکرد درشت‌به‌ریز (Coarse-to-Fine) با تغییر اندازه وکسل از 0.5 به 0.1 متر، بهترین راهکار برای حفظ سرعت و دقت به طور همزمان بود.

- **مقداردهی اولیه ICP:** الگوریتم ICP یک بهینه‌ساز محلی است و به شدت به حدس اولیه وابسته است. اگر دو ابر نقطه در ابتدا بسیار دور از هم باشند یا چرخش زیادی نسبت به هم داشته باشند، الگوریتم بدون یک حدس اولیه مناسب (مانند ماتریس تبدیل اسکن قبلی یا خروجی مرحله Coarse) هرگز به درستی همگرا نخواهد شد.

- **افتادن در Local Minima:** زمانی که آستانه جستجو (Threshold) بیش از حد بزرگ باشد یا حدس اولیه مناسب نباشد، الگوریتم ممکن است ساختارهای اشتباهی را به هم متصل کند (مثلاً تطبیق یک درخت در اسکن اول با درخت دیگری در اسکن دوم). این حالت باعث می‌شود الگوریتم تصور کند به خطای کمینه‌ای دست یافته است (توقف پیش از موعد)، در حالی که از نظر فیزیکی کاملاً در اشتباه است.

- **تاثیر (Scene Geometry):** الگوریتم Point-to-Plane در محیط‌هایی با ساختارهای مسطح و معماری مشخص (مانند دیوارهای ساختمان‌های صنعتی در این دیتاست) بسیار عالی عمل می‌کند، زیرا سطوح می‌توانند روی هم بلغزند تا به بهترین تناسب برسند. اما در محیط‌های کاملاً بدون ساختار (مانند جنگل‌های انبوه با شاخ و برگ تصادفی)، محاسبه بردار نرمال دچار خطا شده و روش Point-to-Point ممکن است پایداری بیشتری از خود نشان دهد.

بر اساس نتایج به دست آمده در این پروژه (تراز کردن جفت اسکن‌ها در بخش اول و نقشه‌سازی محلی در بخش دوم)، روش پیاده‌سازی شده دارای نقاط قوت و محدودیت‌های مشخصی است:

این (Pipeline) در محیط‌های ساختاریافته به دلیل استفاده از روش درشت‌به‌ریز و معیار خطای Point-to-Plane بسیار موفق عمل کرد. تنظیم صحیح آستانه‌ها باعث شد تا الگوریتم با موفقیت بر خطای اولیه غلبه کرده و به امتیاز برآزش بالای ۸۵ درصد و خطای زیر متر دست یابد. نقطه ضعف اصلی این روش، محدودیت در مدیریت حافظه (RAM Exhaustion) هنگام ادغام ده‌ها اسکن است. از آنجا که حجم نقاط در نقشه‌سازی متوالی به صورت تصاعدی بالا می‌رود، در صورت عدم استفاده دوره‌ای از فیلتر کاهش نمونه‌برداری، سیستم با مشکل کمبود حافظه مواجه می‌شود. علاوه بر این، این روش به تنهایی (بدون GPS یا IMU) در برابر حرکات ناگهانی و خشن پهناد به دلیل از بین رفتن هم‌پوشانی بین فریم‌ها، به شدت آسیب‌پذیر است.